Projet : « Very Good tri(p) »

L'étude de l'efficacité d'un algorithme porte souvent sur deux principaux facteurs :

- le temps d'exécution
- et l'espace mémoire nécessaire pour résoudre un problème.

Bien que les performances des ordinateurs ne cessent de croître de manière exponentielle, il est toujours intéressant d'utiliser des algorithmes performants ne serait-ce que parce que la quantité des données que ces algorithmes doivent traiter est aussi en constante augmentation.

Dans ce projet, nous allons nous intéresser aux performances de quatre algorithmes de tri (simples) et essayer de déterminer leur complexité temporelle (qui s'exprime en fonction de n, la taille des données).

Le projet pourra s'écrire en Python ou en GNU Octave. Pour des raisons de simplicité, nous manipulerons des données **entières** à trier.

Fonctions Outils

- Ecrire tout d'abord la fonction **T = Genere_Tableau(n,valmax)** qui génère un tableau de **n** données entières aléatoires entre **0** et **valmax**.
- Ecrire les deux fonctions **Genere_TableauTrie(n, valmax)** qui génère un tableau de n données entières triées du plus petit au plus grand dans l'intervalle [0, valmax] et **Genere_Tableau_Trie_Inverse(n, valmax)** qui génère un tableau de valeurs triées du plus grand élément au plus petit dans l'intervalle [0, valmax].
- Programmer une fonction qui réalise l'insertion à partir d'un tableau de nombres entiers, d'un élément stocké à un indice de position initiale vers un indice de position finale.

Algorithmes de tri

Coder tout d'abord les quatre algorithmes de tri suivants :

- **Tri_Insertion** : Le tri par insertion consiste à classer les deux premiers éléments du tableau. Une fois que les deux premiers sont ordonnés, on prend l'élément qui suit et on le classe à son tour dans ce qui a déjà été classé.
  Pour chaque élément i du tableau, on sait que les éléments de 1 à i-1 sont déjà classés. On va chercher la position j parmi les i-1 premiers éléments de manière à ce que T[i] < T[j]. On insère alors T[i] à la position j.

- **Tri_Selection** : Le tri par sélection consiste à chercher le plus petit élément du tableau et à le placer en première position. Une fois le plus petit élément positionné en première position, on recommence la même opération en commençant à l'indice 2.
  On cherche le plus petit élément compris dans le tableau entre la position 2 et la fin du tableau, puis on place cet élément à la position 2. On recommence ensuite en partant de l'élément 3, 4 et ainsi de suite jusqu'au bout du tableau.

- **Tri_QuickSort** (tri rapide) : Ecrire les fonctions **partitionner**, **tri_rapide** et **tri_QuickSort** définies par :

```
fonction [p, T] = partitionner(tableau T, premier, dernier)
     pivot = T[premier] ;
     p = premier ;
     pour i de premier+1 à dernier
         si T[i] <= pivot alors
             p = p+1 ;
             échanger T[i] et T[p]
         finsi
      finpour
         échanger T[premier] et T[p]
fin

fonction t = tri_rapide(tableau t, premier, dernier)
    début
      si premier < dernier alors
        [pivot, t] = partitionner(t,premier,dernier)
        t = tri_rapide(t,premier,pivot-1)
        t= tri_rapide(t,pivot+1,dernier)
      fin si
    fin
```

```
fonction t= tri_quicksort(tableau t)
    taille  = taille(t);
    t = tri_rapide(t, 1, taille);
fin
```

- **Tri_Shell** : Ecrire enfin la fonction **tri_Shell** définie par :

```
fonction T = tri Shell(tableau T)
     pas = 0
     longueur = taille(T)
     tant que pas < longueur faire
          pas=3*pas+1 ;
     fintantque
     tant que pas !=0 faire
          pas = pas/3
          pour i=pas+1 à longueur faire
               valeur = T[i]
               j = i
               tant que (j>pas) et (T[j-pas]>valeur)
                    T[j] = T[j-pas]
```

```
                    j=j-pas
                fintantque
                T[j]=valeur
            finpour
        fintantque
fin
```

Comparaison des quatre méthodes

Les méthodes de tri vont devoir être évaluées minimalement au moyen de **deux variables** : le nombre d'éléments à trier (aléatoires) et le temps nécessaire pour réorganiser le tableau (utiliser pour ce faire par exemple les fonctions GNU Octave **tic()** et **toc()** ou **cputime()**) ou en Python avec le module time.

Il vous est **demandé** d'évaluer ces quatre méthodes de tri pour des tableaux de taille de **100** à **5 000** éléments en augmentant à chaque fois la taille du tableau de **200 éléments**.

A chaque itération, il s'agira de créer un tableau de la taille demandée avec des valeurs soit générées aléatoirement ou de manière fixe (pour gérer **le meilleur et pire cas**). Le tableau créé devra alors être trié successivement avec les quatre méthodes de tri.

Pour chaque taille de tableau, les tris seront évalués **10 fois** chacun afin d'identifier si le temps d'exécution varie (si c'est le cas, il faudra proposer une solution de gestion de cette variation).

Les résultats d'évaluation seront compilés :

1. sous la forme d'un tableau pour les résultats concernant le temps d'exécution. Ce tableau aura la forme suivante :

Taille Tableau	Temps moyen insertion + min/max	Temps moyen sélection + min/max	Temps moyen QuickSort + min/max	Temps moyen Shell + min/max

   Les temps moyens seront calculés sur les 10 essais effectués pour chaque tri et chaque taille.

2. sous la forme de graphiques représentant le temps nécessaire pour chaque méthode de tri. Le graphique aura comme abscisse le nombre d'éléments du tableau à trier, en ordonnée la variable étudiée (temps) et chaque méthode de tri sera représentée par une courbe de couleur différente avec un texte associé.

Essayer pour chacune des méthodes de déterminer la complexité temporelle pour chacune des méthodes de tri : le temps de tri évolue-t-il de manière linéaire, polynomiale, logarithmique, exponentielle ?

Conclure sur la « *meilleure* » méthode de tri d'éléments. Justifiez vos choix au travers d'exemples et de **critères pertinents**.

Travail à rendre

Vous devrez rendre pour le **13 janvier 2027 dernier délai** par voie électronique (à l'adresse : **Philippe.Truillet@upssitech.fr et Victor.Brehault@irit.fr**) :

- Les fichiers **.m**, python ou **.ipynb** que vous aurez créés. Il vous est conseillé de structurer votre projet pour les fonctions de calcul, pour l'évaluation et la comparaison des méthodes de tri ;
- Un **rapport** décrivant les algorithmes que vous aurez programmés (notamment pour l'évaluation) en expliquant votre démarche de travail.

Une présentation orale de 10 minutes (avec documents pour la vidéo-projection) aura enfin lieu après la fin du projet entre le **18 et 22 janvier 2027**.